

HOPPER INFORMATION SERIES

04 / TEXT CODING

JavaScript API reference

The stable variables and functions available to advanced Hopper Studio programs.

ASYNC + AWAIT

DRONE

VISION

RUNTIME





Your program receives four useful bindings

D drone

Flight, state and accessory commands shared by the real and simulated aircraft.

V vision

Thresholding, object detection, custom classification and AprilTag functions.

R runtime

Keys, drone events, loops, cancellation and cooperative yielding.

C console

log, warn and error lines in the in-app program console.

TOP-LEVEL `await` WORKS

```
await drone.takeOff();
```

Promise-returning calls must be awaited or the main body may finish and auto-land early.



drone: core flight commands

```
await drone.takeOff()
```

Take off; physical controller waits about 3 s.

```
await drone.land()
```

Land; physical controller waits about 5 s.

```
await drone.hover()
```

Zero axes and wait 1 s.

```
await drone.fly(direction, seconds, power)
```

6 directions; seconds ≥ 0 ; signed power clamps -100..100%.

```
await drone.rotate(degrees, direction)
```

clockwise/counterclockwise; timed/open-loop yaw.

```
await drone.flip(direction)
```

forward/backward/left/right; preserve safe clearance.

```
drone.setAxis(axis, power)
```

Persistent pitch/roll/yaw/gaz command until reset.

```
drone.reset()
```

Zero motion axes; does not land.

"altitude" is accepted as an axis alias for gaz vertical power - it is not a height setpoint.



drone: state, timing and accessories

STATE + TIMING

- `await drone.wait(seconds)`
- `drone.getBatteryLevel()` -> number | null
- `drone.isFlying()` / `drone.isLanded()` -> boolean
- `await drone.waitUntilBatteryLevelChanges()`
- `drone.cancelRunFlag` -> boolean

CAMERA + ACCESSORIES

- `await drone.takePicture()` -> session gallery
- `await drone.grabber("OPEN" | "CLOSE")`
- `await drone.fireGun()`
- `await drone.cutoff()` -> emergency only

APP-OWNED - DO NOT USE IN STUDENT PROGRAMS

`disconnect`, `abortRun`, `startRun`, `stopRun`, `landNoWait`, `forceLand`, `manualNudge`, `onTelemetry` and `onEvent` are lifecycle/UI internals. Replacing callbacks can break the app.



vision: binary and COCO functions

BINARY THRESHOLD

- `await vision.scanThreshold(T=60, invert=false)`
- `await vision.seesBinary("white", T, invert, coverage=10)`
- `await vision.binaryCenter("white", T, invert)`
- T and coverage use 0-100 percentages.

COCO OBJECTS

- `await vision.loadObjectModel()`
- `await vision.detectObjects(confidence=0.55)`
- `await vision.seesObject("apple", 0.55)`
- `vision.objectCoordinate("apple", "x", 0.55)`

OBJECT RESULT

```
{ class, score, bbox, frameWidth, frameHeight, centerX, centerY }
```

x/y are -100..100; right/up positive. `objectCoordinate` never scans and returns last known value or 0.



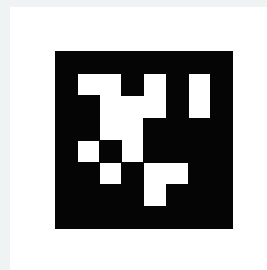
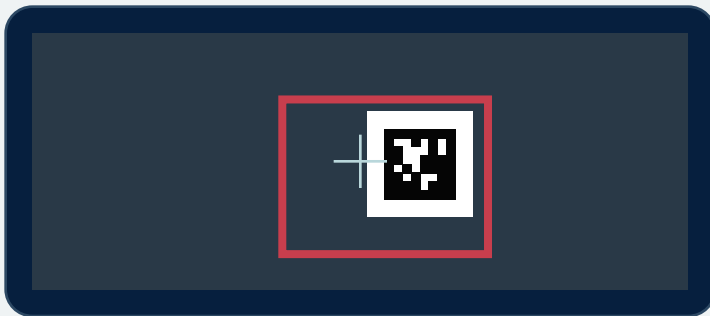
vision: AprilTags and custom models

APRILTAGS

- `await vision.scanAprilTags()`
- `await vision.seesAprilTag("any" | id)`
- `await vision.centerOnAprilTag(drone, id, power, centerSlack, angleSlack, lostSearches)`
- IDs 0-586; 2D image yaw; returns true/false.

TEACHABLE MACHINE

- Load `model.json` + `weights.bin` + `metadata.json` in the UI.
- `await vision.classifyCustomModel()`
- `await vision.seesCustomLabel("landing pad", 0.75)`
- Whole-frame classes; no boxes or x/y location.



tag36h11 ID 0

1 Frame

Fresh camera capture.



2 Inference

Runs on this computer.



3 Decision

Boolean, result list or pose.



runtime and console

```
runtime.registerKey("pressed", "a", async () => {...})
```

Key event; handlers can overlap.

```
runtime.registerDrone("landed", async () => {...})
```

flying, landed, crashed, batteryLevelChanged.

```
runtime.keyIsPressed("ArrowUp")
```

Live Boolean.

```
await runtime.repeatForSeconds(seconds, async () => {...})
```

Sequential iterations with a yield.

```
await runtime.tick()
```

Yield and throw if stopped.

```
runtime.stopped / runtime.hasEvents
```

Read program state.

```
runtime.stop()
```

Unregister/abort only; currently does NOT land.

```
console.log / warn / error(...values)
```

Append to the in-app console.

Use the red UI STOP & LAND for a safe operator intervention.



A complete safe example

```
WHITE-PAPER-LANDING.JS

1  await drone.takeOff();
2  try {
3    while (!runtime.stopped) {
4      const overPaper = await
5        vision.binaryCenter("white", 60, false);
6      if (overPaper) break;
7      await drone.fly("forward", 0.5, 12);
8    }
9  } finally {
10   await drone.land();
11 }
```

WHY THIS PATTERN

- Top-level await keeps order.
- Fresh vision decision after each move.
- Small, conservative command.
- finally sends land after error or break.

REMEMBER

- No GPS or altitude variable.
- Vision can be wrong - test lighting first.
- UI Stop & Land remains the operator control.
- Keep the flight area clear.



Use the documented surface

UNITS AT A GLANCE

- Flight power: percent, recommend 0-100.
- Fly/wait time: seconds.
- Rotation: degrees + direction.
- JS confidence: fraction 0-1.
- Block confidence/threshold/coverage: percent 0-100.
- Vision x/y: -100..100.

SOURCES AND FURTHER READING

Shared drone controller contract

lib/drone.ts in the Hopper Studio project

Vision runtime

lib/vision.ts and lib/apriltags.ts in the Hopper Studio project

Program runtime and app execution

lib/runtime.ts and components/HopperStudio.tsx

AWAIT FLIGHT + VISION CALLS

Use documented members. Treat browser globals and app internals as implementation details.